Week 1 Concept Inventory and Build Plan

- Concept Inventory
- Weekly Throughline
- What I Actually Learned
- What Is Still Shallow or Unclear
- Concept Connections
- One Build To Make This Real
- Why This Build
- Coverage Estimate
- Concept-to-Build Mapping
- What This Still Does Not Cover
- Scope
- Execution Plan
- Definition of Done

Concept Inventory

Core Technical Primitives - `np.array()` with dtype specification at creation - `np.zeros()`, `np.ones()`, `np.full()` - `np.identity()`, `np.eye()` (square only vs square or non-square) - `np.random.rand()`, `np.random.randint()` - `np.arange(start, stop, step)` - `np.empty_like(x)` - `np.genfromtxt()` for file I/O - `np.matmul()` / `@` operator - `np.dot()` / `@` producing a scalar from two vectors - `np.add()`, `np.subtract()`, `np.multiply()`, `np.divide()`, `np.sqrt()` - `np.min()`, `np.max()`, `np.sum()` with axis argument - `np.any()`, `np.all()` with boolean conditions - `np.tile(v, reps)` - `reshape()`, `.T`, `.copy()`, `astype()` - `vstack()`, `hstack()`

Mechanics and Properties - Fixed types: why NumPy arrays store one type vs Python lists storing pointers - Contiguous memory: elements adjacent in RAM enabling sequential reads - SIMD vector processing: named, not yet operational - Cache utilization: named, not yet operational - `ndim`, `shape`, `dtype`, `itemsize`, `size`, `nbytes` - `nbytes = size * itemsize` as a concrete formula - 3D shape convention: (blocks, rows per block, columns per row) - Row/column slicing: `a[0, :]`, `b[:, 1]` - Step slicing: `b[0, 1:6:2]` - Integer array indexing - Boolean masking: `a[a > 2]` - Matrix-shape compatibility rule: columns of A must equal rows of B - Broadcasting: arithmetic across different-shaped arrays without copying data - Axis-based operations: summing along axis=0 vs axis=1 produce different shapes - Scalar vs element-wise vs matrix multiplication as three distinct operations

Python Containers - Lists: ordered, mixed types - List comprehension syntax - Dictionaries: key-value pairs - Sets: unordered, unique elements - Tuples: ordered, immutable, valid as dict keys

Systems Concepts - Core challenges: scalability, consistency, reliability, maintainability - CAP theorem: Consistency, Availability, Partition Tolerance - Network partition: the event that forces a tradeoff between C and A - OLTP: low-latency, point queries, the operational write layer - OLAP: large-aggregate reads, analytical use, should not run on the operational store - Data Lakehouse: hybrid approach (light in notes) - ETL: Extract, Transform, Load - Data pipeline: raw ingestion through transformation to storage - Sushi principle: raw data preserves optionality for downstream consumers - Data lake vs data warehouse - System of Record: authoritative source, updated first - Derived Data Systems: caches, analytical tables, anything computed from source - Reverse-ETL: feeding analytical output back into operational systems - Feature engineering: transforming raw records into model-ready numerical features - Cloud vs self-hosting: expertise vs control tradeoff - Cloud-native: architecture designed to exploit cloud services - Named examples: MySQL/Postgres/MongoDB (self-hosted OLTP), Aurora/Spanner (cloud OLTP), Snowflake/BigQuery (cloud OLAP)

Roles and Components - Backend engineers: build and operate OLTP systems - Data engineers: own ETL, integrate operational and analytical systems - Analytics engineers / data scientists: consume derived data for modeling and reporting

Open Questions / Future Threads - SIMD: can name, cannot explain from first principles yet - Cache utilization: same - CAP theorem: defined, no worked example of the tradeoff in a real system - `tokenizer?`: flagged as open question in source notes - `canonical?`: same - HTAP, cloud/self-hosting architecture details, TFX/Kubeflow/MLflow: deferred

---

Weekly Throughline

Week 1 was about learning to think in structured transformations. On the NumPy side: replacing loop-by-loop thinking with operations that treat entire arrays as the unit, and understanding why that difference runs deeper than syntax. On the systems side: understanding that data doesn't live in one place, it flows from authoritative operational sources through transformation into derived analytical forms, and that architecture is the set of decisions governing that flow. These two threads are not parallel: the NumPy operations you're learning are the compute primitives for the transformations that data architecture describes.

What I Actually Learned

Foundations - NumPy arrays differ from Python lists not just in speed but in memory model: fixed types, contiguous storage, SIMD-readiness. The performance story is a chain, not a single fact. - Array properties (ndim, shape, dtype, itemsize, size, nbytes) form a complete inspection vocabulary. `nbytes = size * itemsize` is a formula you can derive, not just recall. - The 3D shape convention (blocks, rows, columns) is understood, with at least one real correction from intuition to actual output this week.

Mechanisms - Matrix multiplication has a hard compatibility rule: columns of A must equal rows of B. You know why this is true geometrically, not just that it is. - Dot product (scalar output) and matmul (matrix output) are distinct operations. - Axis-based statistics produce different output shapes depending on which axis you collapse. - Copy vs reference: assignment points to the same memory, `.copy()` does not. - Boolean masking filters arrays without loops.

Practical Operations - Full range of array creation functions and when each applies. - Slicing including step-based syntax. - Reshape, transpose, stack, tile, type conversion all noted and available. - File I/O via `np.genfromtxt()`.

Systems Ideas - OLTP and OLAP are purpose-differentiated systems. Running analytical scans against an operational store degrades performance for real users. - ETL is the movement of data between those purposes with transformation in between. - System of Record and Derived Data are a design pattern you can apply to any architecture. - Reverse-ETL closes the loop from analytics back to product. - The Sushi principle is a concrete argument for not over-transforming raw data before you know what downstream consumers need.

What Is Still Shallow or Unclear

- **SIMD and cache utilization**: Can name these. Cannot yet explain what SIMD does to a vector instruction at the hardware level, or what makes a memory access pattern cache-friendly. This is Week 2's checkpoint question and needs first-principles reasoning, not lookup.
- **Broadcasting**: Defined as a mechanism, not yet exercised. The shape-prediction experiments the Q2 plan calls for have not happened yet.
- **CAP theorem**: Definition is present. No worked example of what "choosing consistency over availability" looks like during an actual network partition in a real system.
- **np.eye vs np.identity**: The distinction is noted. Unclear if verified in code.
- **np.any() / np.all()**: Listed, no evidence of use beyond the listing.
- **Reverse-ETL**: Defined cleanly, no worked design reasoning about when you would or wouldn't use it.
- **Feature engineering**: Definition only. No connection yet to what it will concretely mean when preparing inputs for a neural network.
- **`tokenizer?` and `canonical?`**: Open questions from the source notes, unresolved.
- **Roles**: Can name and roughly describe each. Not yet connected to concrete ownership boundaries or decisions in a real architecture.

Concept Connections

The performance story is a chain: fixed types enable contiguous memory, contiguous memory enables cache-efficient sequential reads, cache efficiency is what SIMD needs to process multiple

elements per CPU instruction. “NumPy is faster” is not the idea. That chain is.

NumPy matrix operations are the forward pass of a neural network. Every `np.matmul(inputs, weights)` you write this week is the same computation happening in a layer. The column-row compatibility rule governs whether a layer’s output is the right shape to feed the next layer’s input. This is not a future fact. It is already true of the code you are writing.

OLTP/OLAP, ETL, and Derived Data are the systems-layer version of something you will do in every ML project: raw records from an operational source get transformed into feature matrices a model can consume. Feature engineering is the ML name for that transformation. Reverse-ETL is what happens when the model’s output score gets pushed back to the product layer, e.g. a recommendation ranking surface.

Broadcasting is a memory efficiency mechanism: instead of copying a smaller array to match shapes, NumPy expands it virtually during the operation. This will matter when you apply bias vectors across entire batches of inputs in your neural network.

One Build To Make This Real

Build **`user-activity-pipeline`**: a Jupyter Notebook that plays the role of a junior data engineer handed raw user event data from a production database, transforms it into analytics-ready features using pure NumPy, computes an engagement score per user via matrix multiplication, writes it back as a product-facing output, and documents every step the way you would in a real job.

Why This Build

A single matrix multiplication script exercises maybe 10% of this week’s notes. The week covered two large domains with real overlap, and the build should force that overlap into view. This specific framing (user activity data, analytics pipeline, engagement score, reverse ETL) is not contrived: it is a pattern you will encounter in a real data or ML engineering role within the first month. The framing also does something a generic script cannot: it gives every NumPy operation a job-relevant reason to exist. Normalization is not a math exercise. It’s what sklearn’s `StandardScaler` does internally before passing data to a model. Matmul is not a NumPy function. It’s what a deployed linear model’s `predict()` call runs.

Coverage Estimate

Full build: approximately 85-90% of NumPy mechanics. Systems concepts: approximately 60-65%. OLTP/OLAP, ETL, system of record, derived data, reverse-ETL, and roles are all present in the

framing or code. CAP theorem, HTAP, cloud/self-hosting architecture, and named ML pipeline tooling are deferred honestly in the open questions panel.

Concept-to-Build Mapping

Concept	Where It Appears
Fixed types, contiguous memory	Module 0: performance demo measuring a Python loop vs np.sum on 1M elements, with printed explanation
SIMD, cache utilization	Module 0: written explanation of why the timing gap exists at the hardware level
ndim, shape, dtype, itemsize, size, nbytes	Module 1: print all six properties, with a one-line comment explaining each
1D, 2D, 3D arrays	Module 1: create one of each, verify the 3D shape convention
np.zeros, np.ones, np.full	Module 1: create a zeroed placeholder output table, a ones-based mask, a full sentinel value
np.identity, np.eye	Module 1: create both, print comparison, explain the non-square capability of eye
np.random.rand, np.random.randint	Module 1: generate the synthetic user_events dataset
np.arange	Module 1: generate user_id sequence
np.empty_like	Module 1: create a matching container for the output array
np.genfromtxt	Module 1: load a small CSV if available, or note where the SQL query would go in production
Integer array indexing	Module 2: select specific user record indices for inspection
Row/column slicing, step slicing	Module 2: extract individual feature columns from the event matrix
Boolean masking, np.any, np.all	Module 2: filter users whose purchase_amount exceeds threshold, validate with np.any/np.all

Concept	Where It Appears
astype	Module 2: cast integer columns to float64 before normalization
Normalization (mean, std, axis=0)	Module 2: “this is what StandardScaler does under the hood” - normalize all feature columns without loops
Element-wise multiply, divide, sqrt	Module 2: compute derived columns (e.g. pages per second)
.copy()	Module 2: copy raw data before mutation, with a comment explaining why
reshape	Module 2: reshape a feature vector before matmul
vstack, hstack	Module 2: assemble the final feature matrix from individual columns
np.tile	Module 2: replicate a bias row across all 500 user records
np.sum, np.min, np.max along axis	Module 3: per-region aggregation (total revenue, avg session duration, max pages viewed)
.T (transpose)	Module 3: transpose weight vector for matmul scoring
np.matmul / @	Module 3: compute per-user engagement score as (feature_matrix @ weights)
np.dot	Module 3: compute a single user’s score as a scalar, contrasted with matmul
Reverse-ETL	Module 3: write the score column back as an operational-facing output column - note where Census or Hightouch would do this in production
Feature engineering	Module 2/3: the transformation from raw events into model-ready numerical columns
Broadcasting	Module 4: three shape experiments with predictions written as comments before each run

Concept	Where It Appears
Python lists, dicts, tuples, sets	Setup: config dict for column names, tuple for shape assertions, set for unique region IDs, comprehension for column label generation
List comprehension	Setup: generate column header labels
OLTP / low latency / point queries	Module 5: framing of the raw dataset as “what lives in PostgreSQL” - note why you can’t run aggregations on it directly
OLAP / large aggregates	Module 5: framing of Module 3 output as “what Snowflake or BigQuery would serve”
ETL	Narrative connecting Modules 1, 2, and 3 with section headers
System of Record vs Derived Data	Module 5: named explicitly with the user_events as source and the score table as derived
Sushi principle	Module 5: markdown explanation of why the raw user_events array is preserved alongside the transformed feature matrix
Data lake vs data warehouse	Module 5: one-paragraph distinction inside the systems panel
Backend / data / analytics engineer roles	Module 5: who owns what - backend owns user_events, data engineer owns Modules 1-3, analytics engineer consumes Module 3 output
CAP theorem, network partition	Module 6 (open questions panel): defined, flagged as not yet demonstrated with a worked example
Cloud vs self-hosting	Module 6: named as a future thread, with the comparison table from notes preserved
tokenizer?, canonical?	Module 6: carried forward explicitly as unresolved margin questions
SIMD / cache (if performance demo doesn’t fully resolve it)	Module 6: flagged if still not operational after Module 0

What This Still Does Not Cover

- **CAP theorem / network partition**: Could be simulated - two functions returning different values depending on a `partition=True` flag, forcing you to choose which to trust. Not included because the simulation would be artificial without real distributed systems context. Better addressed when DDIA Chapter 5 (Replication) arrives in Week 5.
- **HTAP**: Not in notes with enough depth to build against honestly. Deferred.
- **Cloud vs self-hosting architecture**: Architectural tradeoff, not implementable in a local notebook. Represented in the open questions panel with the comparison table from notes preserved.
- **TFX/Kubeflow/MLflow**: Not in notes. Relevant when Q3 reaches model serving and orchestration.
- **`tokenizer?` and `canonical?`**: Genuinely open. Cannot resolve without more context. Carried forward explicitly.

Scope

Modules: - Module 0: Performance demo (Python loop vs NumPy on 1M elements) - Module 1: Dataset setup and array inspection - Module 2: ETL transformation and feature engineering - Module 3: OLAP aggregation, matmul scoring, reverse-ETL output - Module 4: Broadcasting experiments - Module 5: Systems concepts panel (markdown only) - Module 6: Open questions panel (markdown only)

Minimum honest version: Modules 1, 2, and 4. Covers NumPy core and broadcasting. Missing the production framing and systems depth.

Full version: All seven modules. This is what earns the 85-90% coverage estimate and what you'd actually show in an interview or portfolio.

Stretch version: Add a Matplotlib plot of the engagement score distribution across regions. Add a timed comparison cell in Module 0 that prints the speedup ratio, not just the times. Add a markdown cell at the end that describes in one paragraph how you'd replace each module with a production tool (PostgreSQL query, dbt transform, Airflow DAG, reverse-ETL via Census).

What not to build yet: No Pandas, no sklearn, no visualization beyond one optional chart, no external API calls, no UI. Pure NumPy for all operations.

Execution Plan

1. **Setup**: Generate a realistic synthetic `user_events` dataset - 500 rows, columns: `user_id` (int), `session_duration_sec` (float), `pages_viewed` (int), `purchase_amount` (float), `region_id` (int, 0-

4). Save to CSV with `np.savetxt`. This is your fake PostgreSQL export.

2. **Module 0**: Time a Python loop summing 1M random floats vs `np.sum()`. Print the ratio. Write a markdown cell below it explaining fixed types, contiguous memory, and SIMD as a chain, not a list.
3. **Module 1**: Load the CSV with `np.genfromtxt`. Print all six properties. Create one 1D, 2D, and 3D array. Print shapes. Add a markdown annotation: "In production: this genfromtxt call is a SQLAlchemy query to a PostgreSQL OLTP database."
4. **Module 2**: Write a normalize function (subtract column mean, divide by column std, axis=0 - no loops). Write a comment: "This is what sklearn's StandardScaler does internally." Extract feature columns via slicing. Compute a derived column (pages per second). Apply a boolean mask for high-value users. Validate with `np.any()` and `np.all()`. Assemble the final feature matrix with hstack. Add a `.copy()` before any mutation with a comment explaining why.
5. **Module 3**: Aggregate by region using `np.unique` and axis-based sum and mean. Print the OLAP-style report. Define a weight vector manually (your first "model"). Compute per-user engagement scores with `np.matmul(feature_matrix, weights)`. Write scores as a new column back into the array. Add a markdown annotation: "In production: this score gets written back to PostgreSQL by a reverse-ETL tool like Census or Hightouch, and surfaces in the product as a recommendation ranking."
6. **Module 4**: Run three broadcasting experiments. Before each one, write a comment predicting whether it works and the output shape. Run it. Correct any wrong predictions in a markdown cell below.
7. **Modules 5 and 6**: Write the systems concepts panel in markdown. Write the open questions panel. Name at least three things you genuinely cannot yet explain. This is not a shame list. It is the thing a senior engineer would ask you about in an interview.

Definition of Done

You are done when: - The notebook runs clean, top to bottom, no errors - You can explain out loud what normalization along `axis=0` does, why axis matters, and what happens to the shape of the output - without looking at the code - You can state the output shape of `np.matmul((500, 4), (4, 1))` before running it and explain why that rule exists - You can explain the performance demo result as a hardware-level chain (fixed types -> contiguous memory -> cache -> SIMD), not just "NumPy is faster" - You can describe each module in one sentence the way you'd describe it to a senior engineer: "This is the ETL step. It takes the raw OLTP export, normalizes the features, and assembles the matrix the model will score against." - The open questions panel has at least three items you cannot yet fully explain, named honestly - You can walk someone through the full notebook as a production story: here is the operational source, here is the transformation, here is

the analytical output, here is where the score feeds back into the product - and you can name which real tool would replace each module at scale